Chapter 3

Monitors and blocking operations

This chapter introduces the concepts of monitor, conditional critical section and condition variable. It shows how to write a class in Java so that the operations (the methods) preserve the integrity of its shared state (the fields) and so that an operation can block until the state makes it meaningful for the operation to proceed.

3.1 A simple monitor

In the Tivoli gate example, the turnstile at an entry gate simply counts how many visitors have entered the Tivoli Gardens. Let us add also turnstiles at exit gates to also count how many visitors leave Tivoli, and maintain a field `present` which is the number of visitors inside. If we use explicit locks of class ReentrantLock, we might have this code:

```
class Tivoli2 {
  private final Lock lock = new ReentrantLock();
  private long present = 0;

  public void enter() {
    lock.lock();
    present++;
    lock.unlock();
  }

  public void exit() {
    lock.lock();
    present--;
    lock.unlock();
  }
}
```

Due to the lock taking by both methods, this correctly counts the number of visitors currently present inside Tivoli. However, we may additionally wish that an entry gate admits a new visitor only when the park is not full, say, when less than 15,000 visitors are present inside Tivoli. Otherwise the entry gate should block and wait for some visitors to leave via the exit gates, and only then allow a new visitor to enter.

The conventional way to do this in Java is via *condition variables*. Whereas the purpose of

taking a lock (in the Tivoli gate example) is to create a critical section, the purpose of a lock together with a condition variable is to create a *conditional critical section*: a critical section that is entered only when a condition is satisfied. In the Tivoli case, the condition is that the amusement park is non-full (here putting the capacity at 5 visitors instead of 15,000):

```
class Tivoli3 {
  private final int CAPACITY = 5;
  private final Lock lock = new ReentrantLock();
  private final Condition nonFull = lock.newCondition();
  private long present = 0;

  public void enter() {
    lock.lock();
    while (present >= CAPACITY)
      try { nonFull.await(); } catch (InterruptedException exn) { };
    present++;
    System.out.printf("Entering... now %d present\n", present);
    lock.unlock();
  }

  public void exit() {
    lock.lock();
    present--;
    System.out.printf("Exiting...  now %d present\n", present);
    nonFull.signalAll();
    lock.unlock();
  }
}
```

If we create and run a thread repeatedly calling `enter` and another repeatedly calling `exit` we might get a trace like this:

```
Entering... now 1 present
Exiting...  now 0 present
Entering... now 1 present
Entering... now 2 present
Entering... now 3 present
Exiting...  now 2 present
Entering... now 3 present
...
Entering... now 5 present
Exiting...  now 4 present
Exiting...  now 3 present
Entering... now 4 present
```

The Tivoli3 class creates a condition variable `nonFull` from the lock object. The condition variable provides a mechanism for a thread to await the fulfillment of a condition, and for other threads to signal that the condition may now be fulfilled, so it is worth testing it again. In the Tivoli3 class, a thread calling the `enter` method checks whether Tivoli is full (`present >= CAPACITY`) and if so, awaits the `nonFull` condition; this puts the thread in an "awaiting `nonFull`" pool of inactive threads. Another thread calling the `exit` method will decrement `present` and signal all threads in the "awaiting `nonFull`" pool that it is worth waking up

and evaluating `(present >= CAPACITY)` again. Despite its name, a condition variable such as `nonFull` cannot be used as a logical condition in an if- or while-statement: its purpose is only to orchestrate that threads block while awaiting a change, and to signal to such blocked threads that something might have changed.

Note that this happens in a while-loop; an if-statement would not be adequate for this purpose. There are several reasons for this. Although the `exit` method decreased the `present` count, there may be multiple threads trying to enter, and only one of them should succeed. Another more technical reason is that the operating system may produce spurious signals, causing an entering thread to awake even though the state did not change.

A thread calling `await` or `signalAll` on a condition variable must hold the lock associated with the condition variable. As you can see, this is the case in Tivoli3. A subtle point is that when a thread calls `await`, it implicitly temporarily releases the lock; otherwise no other thread could call `exit` and let visitors out of the park. When the awaiting thread gets signalled, and the `await` call returns, the thread obtains the lock again before it can continue executing the method body, and holds the lock until the `unlock` operation (or until it calls `await` again).

Finally, we have enclosed the `await` call in a try-catch block. This is because the call is blocking and the blocked thread `u` may be interrupted (by another thread calling `u.interrupt()`) while in the awaiting pool, in which case it throws InterruptedException when awaking again. The try-catch block above just ignores the exception, which is bad practice, but this is not important to the example.

3.2 Monitors, definition and history

The Tivoli3 class implements a *monitor*: *a shared state and meaningful operations on the state that protect it from corruption, even when performed concurrently.* A monitor protects the state from meaningless updates by having private fields, by all public operations being methods taking the same lock before accessing the state, and by using condition variables to allow a method to proceed only when the state makes that meaningful. More precisely, the condition variable allows a method that cannot proceed to block (using no resources while blockes), rather than spin or busy-wait (potentially wasting much work and electricity).

In the Tivoli3 class, the state is the `present` field which is private, the operations are the public methods `enter` and `exit` which both take the same lock, and the `nonFull` condition variable is used by the `enter` method to proceed only when more visitors can be admitted.

The concept of monitor was invented by Per Brinch Hansen who in 1973 gave this general definition in the context of concurrency: "*I will use the term monitor to denote a shared variable and the set of meaningful operations on it*" and many examples [2, page 121]. He also introduced the concept of conditional critical section (then called "region") [2, page 98]. Brinch Hansen built the concept of monitor on the then newly invented concept of class from Simula 67, the ancestor of all object-oriented languages [2, p. 228]. The Java programming language could have included "monitor" as a special kind of class, but did not, so we need to implement a monitor by careful use of private modifiers, public methods that take the same lock, and condition variables as exemplified in this chapter.

It was Tony Hoare, the inventor of quicksort and much else, who popularized Brinch Hansen's monitor concept in a 1974 paper [6]. Hoare's paper offered design advice that still applies today, including "*the writer of a monitor should declare a variable of type condition for*

each reason why a program might have to wait" [6, page 550]. We shall see this in the bounded buffer examples in sections 3.3 and 3.4 below.

In the literature there are several variations on the definition of monitor. For instance, the 2012 textbook by Herlihy and Shavit presents some code and says "*This combination of methods, mutual exclusion locks, and condition objects is called a monitor*" [5, page 181], describing a monitor more by how it is implemented than what its purpose is.

Brinch Hansen himself uses the term monitor also in the more specific sense of an operating system kernel [2, page 239]. Although somewhat confusing, this is meaningful since the kernel protects and maintains the operating system state by allowing only meaningful operations on it.

The official Java Language Specification says: "*Each object in Java is associated with a monitor, which a thread can lock or unlock. Only one thread at a time may hold a lock on a monitor*" [4, section 17.1]. This sounds as if a monitor admits only lock and unlock operations, but in reality all Java objects also admit condition variable operations called `wait` and `notifyAll`, doing exactly the same as the `await` and `signalAll` operations on class Condition, but operating on the single condition variable implicitly associated with an object. Other Java literature uses the term *intrinsic lock* for what the language specification calls a monitor. Therefore perhaps "intrinsic monitor" would have been the most precise term for the Java concept, but nobody uses it.

The 2006 Java concurrency textbook by Goetz and others defines the *Java monitor pattern* like this: "*An object following the Java monitor pattern encapsulates all its mutable state and guards it with the object's own intrinsic lock*". This is different from our definition (page 3.2) because it leaves out the use of condition variables and blocking. The book also says "*The Java monitor pattern is merely a convention; any lock object could be used to guard an object's state so long as it is used consistently*" [3, pages 60–62], still leaving out the condition variable aspect. A footnote goes on to admit "*The Java monitor pattern is inspired by Hoare's work on monitors (Hoare 1974), though there are significant differences between this pattern and a true monitor*" [3, page 60], where the most interesting difference is the lack of mention of condition variables (although it also ignores Brinch Hansen).

3.3 A single-element buffer as a monitor

Obviously, the Tivoli example is a toy example. For instance, anybody can exit at any time; the exit method is allowed to make the number of present visitors negative.

A more realistic example is a *bounded buffer* that can hold a single integer. This is useful when we have a thread producing a sequence of results and another thread consuming them, for instance, outputting them to a file. Such a buffer has a `put` method that puts a value into the buffer, blocking if the buffer is full, and a `get` method that takes a value out of the buffer, blocking if the buffer is empty. The `put` method is very similar to Tivoli's `enter` method, and the `get` method is similar to the `exit` method except that the `get` method can block.

To make this work correctly, we need two condition variables, following Hoare's advice in section 3.2: one for awaiting the non-fullness of the buffer and one for awaiting the non-emptiness of the buffer. This is implemented in the class SingleIntBuffer:

```
class SingleIntBuffer {
  private final Lock lock = new ReentrantLock();
```

```
  private final Condition nonEmpty = lock.newCondition(),
                          nonFull  = lock.newCondition();
  private int contents;
  private boolean full = false;

  public void put(int v) {
    lock.lock();
    while (full)
      try { nonFull.await(); } catch (InterruptedException exn) { };
    full = true;
    contents = v;
    nonEmpty.signalAll();
    lock.unlock();
  }

  public int get() {
    try {
      lock.lock();
      while (!full)
        try { nonEmpty.await(); } catch (InterruptedException exn) { };
      full = false;
      nonFull.signalAll();
      return contents;
    } finally {
      lock.unlock();
    }
  }
}
```

When field `full` is false, the buffer is empty; when it is true, field `contents` holds the single value in the buffer. The `put` method tests whether the buffer is full, and if so it blocks by awaiting that the buffer has (maybe) become non-full; otherwise (if non-full) it sets `full` to true, sets `contents` to `v`, and signals any threads that await that the buffer is non-empty.

The `get` method symmetrically tests whether the buffer is non-full (that is, empty), and if so it blocks by awaiting that the buffer has (maybe) become non-empty; otherwise it sets `full` to false, signals any threads that await that the buffer is non-full, and returns the buffer's contents.

It does not matter whether `signalAll` is called before or after updating the `full` field, because none of the signalled threads could acquire the lock until this thread has released it, and therefore they also cannot read the `full` field until then.

3.4 A multi-element buffer as a monitor

A bounded buffer that can hold a given number of integers can be built using exactly the same condition variables and in exactly the same way as the single-element buffer, only the buffer storage is an integer array and the indexing into that array is a little intricate. The array is used in a cyclic manner, so after putting something in the last array position one continues by putting something in the first array position (if available).

```
class MultiIntBuffer {
```

```
  private final Lock lock = new ReentrantLock();
  private final Condition nonEmpty = lock.newCondition(),
                          nonFull  = lock.newCondition();
  private final int capacity;
  private final int[] contents;

  private int itemCount, putIndex;

  public MultiIntBuffer(int capacity) {
    this.capacity = capacity;
    this.contents = new int[capacity];
  }

  public void put(int v) {
    lock.lock();
    while (itemCount == capacity)
      try { nonFull.await(); } catch (InterruptedException exn) { };
    contents[putIndex] = v;
    putIndex = (putIndex+1) % capacity;
    itemCount++;
    nonEmpty.signalAll();
    lock.unlock();
  }

  public int get() {
    try {
      lock.lock();
      while (itemCount == 0)
        try { nonEmpty.await(); } catch (InterruptedException exn) { };
      nonFull.signalAll();
      return contents[(putIndex+capacity-itemCount--) % capacity];
    } finally {
      lock.unlock();
    }
  }
}
```

The bounded buffer is empty if `itemCount` is 0 and full if it equals `capacity`. The next place to put something (if non-full) is at array position `putIndex`, and the next place to get something (if non-empty) is from array position `(putIndex+capacity-itemCount) % capacity`.

The latter expression basically subtracts `itemCount` from `putIndex`, modulo `capacity`: when there are `itemCount` items in the buffer, the get position is `itemCount` positions before the put position, modulo the array length.

In the single-element buffer, the "empty" and "full" states were each other's negation. This is not the case for the multi-element buffer, but otherwise their synchronization and condition variable logic is exactly the same, as you can verify by comparing the `put` and `get` methods in the two classes.